OPTIMA

AUTOMATED STUDY FLOW

DEVELOPMENT DIARY

Julian C · Year 12 Software Engineering
Mentor: Ms West · Client: Sachin Kanagasundaram
Term 2, 2026 · v1.0 · 29 May 2026

Development Diary

A log of the design decisions, breakthroughs and stumbles taken across roughly five weeks of building **Optima — Automated Study Flow**. Entries are dated, written in plain English, and grouped by week so the picture of how the application came together can be followed end-to-end.

CONTENTS

Week 1 — Foundations

The goal for the week is to re-read the planning documentation, confirm the tech stack (programming languages, frameworks, etc.), and ship something demonstrable enough to present to the client by Sunday.

PROJECT KICK-OFF

Re-Reading the Specifications

Mon 13 Apr 2026

Opened the planning documentation created for Task 1 and reviewed it from start to finish. I was about to begin typing immediately but ultimately decided to pull back and plan out the main domains of the application on paper first: User, Subject, Constraint (fixed item in the fortnightly rotation), Assignment, ScheduleBlock, Preferences, and an authentication record. Confirmed with the client via text that the 14-day schedule (not the 7-day one) was the one he wanted, as PMH uses it. He sent a photo of his timetable to make sure that his schedule would be able to fit in the application and so that I would have the day numbering right.

DOMAIN-MODEL SKETCH (TRANSCRIBED)

```
User
preferences        :        Preferences       (theme, contrast, focus, zoom)
subjects           :        Subject           (id, name, colour)
constraints[]      :        Constraint        (day_of_fortnight 0-13,
                                              start_time, end_time,
                                              subject_id, is_study_period)
assignments[]      :        Assignment        (subject_id, due_date,
                                              weighting, weighting_hours,
                                              progress
schedule[]         :        ScheduleBlock     (date, start, end,
                                              assignment_id, subject_id)
```

DECISIONS MADE

+ **Python** as the implementation language, as it is a form of structured programming and is also what I'm most fluent in.
+ **Flask + Pywebview** for the GUI. Flask allows the routing and Jinja templates that I had recently learnt, and Pywebview is able to put the app inside a real, natively running application window so it doesn't appear as a website. This allows it to have the best of both, being a website and appearing as an application, and allows it to be framed as a desktop application.
+ **Local JSON** for persistence. By this, the user can work offline, with no server or internet required. The planning brief is explicit that the client has stated a need for something that can be opened offline on their MacBook.

NEXT SESSION

+ Scaffold the folder layout, get Flask running and push v0.0 to the local repository.
+ Set up version control before any code begins to be written.

VERSION CONTROL

Picking a Version Control Approach before beginning

Mon 13 Apr 2026 (evening)

From experience, setting up Git later usually ends with three days of work uncommitted and a sinking feeling when something breaks, or files are accidentally deleted. As such, I decided to settle this on the first day.

OPTIONS CONSIDERED

Approach	Pros	Cons	Verdict
Git + GitHub (private repo)	Industry standard. Branches per prototype. Free remote backup. Familiar workflow.	Need a GitHub account (already have), school Wi-Fi sometimes doesn't work.	**Picked.**
Git, local only (no remote)	No network friction.	If my laptop dies, the whole project dies with it.	Rejected.

Zip-the-folder snapshots	Zero tooling required. Easy to email.	No differentiation, no history, and no recovery can be disastrous if something is overwritten.	Rejected.
Dropbox version history	Automatic. Recovers any file.	Not branch-aware. no commit messages.	Rejected.

THE WORKFLOW BEING KEPT TO

+ **Branch per prototype.** main is always the latest working state; proto/v0.1, proto/v0.2, and proto/v0.3 are tagged snapshots that are copied out into prototypes/<version>_<date> for easier readability.
+ **Committing small/often.** Targeting every commit to be one logical change with a few-line subject and a body that explains why (the diff already explains what changed).
+ **Never commit sensitive info.** Add data/users/*.json and ./venv to .gitignore immediately before there is anything in them.
+ **Tag releases:** git tag v0.1 -m "prototype 0.1 sent to evaluators for feedback", so in the future, I will be able to see what an evaluator saw in previous versions.

```
# initial .gitignore
.venv/
__pycache__/
*.pyc
data/users/*.json
.DS_Store
*.log
```

WHY THIS MATTERS

The three prototype folders are visible artefacts of this workflow, where each one is a snapshot of main on the day that the prototype was sent to the evaluators (exactly what they reviewed). The diary entries dated against those release days line up with the same commits.

REPOSITORY & ENVIRONMENT

Scaffolding the project

Tues 14 Apr 2026

Spent the first hour figuring out pip3. The first few attempts were unsuccessful when trying to use pip until further research was able to enlighten me about pip3 (similar to how 'python3' is used to execute .py files instead of python). Python 3.13 on macOS also now warns about installing into the system site-packages, so I set up a virtual environment in .venv/ and pinned Flask 3.0 to requirements.txt.
Once that was tidy, the actual scaffolding was quick. An app/package with empty stubs for auth.py, models.py, storage.py, and main.py, a templates/ folder, and a run.py launcher.
Was able to test with "hello world" on http://127.0.0.1:5050 by 5 pm.

PROBLEMS ENCOUNTERED

+ PyWebView crashed the first three times I had imported it. Turns out that on macOS, you have to call webview.start() from the main thread only, so Flask has to be the background thread and not the other way around.

NEXT SESSION

+ Login and signup forms. No hashing has yet been implemented; I just need to get the screens visible as mockups.

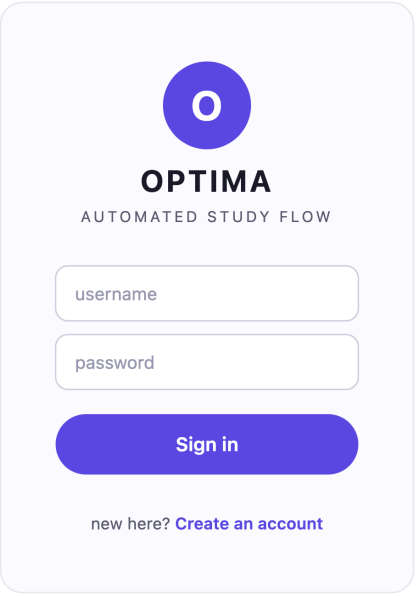
LOGIN UI

First Pass at Login Screen

Wed 15 Apr 2026

Built the static markup for login.html and signup.html. The client and I had previously agreed in planning interviews that the signup should be friction-free, without email or captcha. As such, I have just limited it to *username, password,* and *confirm password*. This aesthetic directly borrows from the storyboard in the planning documentation (with a large icon, ample whitespace, and the primary action as the filled purple pill button). I left the inputs with generic username/password placeholders instead of a hard-coded example name because the planning doc specifies that the product should appear polished and not a private build for a single person.

WIREFRAME (LOGIN.HTML BEFORE CODING)



Login wireframe drawn in Adobe Illustrator before coding the markup.

MARKUP COMMITTED (EXCERPT)

```
<form method="post" action="/login" class="auth-card">
  <input name="username" placeholder="username" required autofocus>
  <input name="password" placeholder="password" type="password" required>
  <button class="btn primary">Sign in</button>
  <a href="/signup" class="link-muted">Create an account</a>
</form>
```

NEXT SESSION

+ Dashboard skeleton so that the post-login state isn't a blank page.

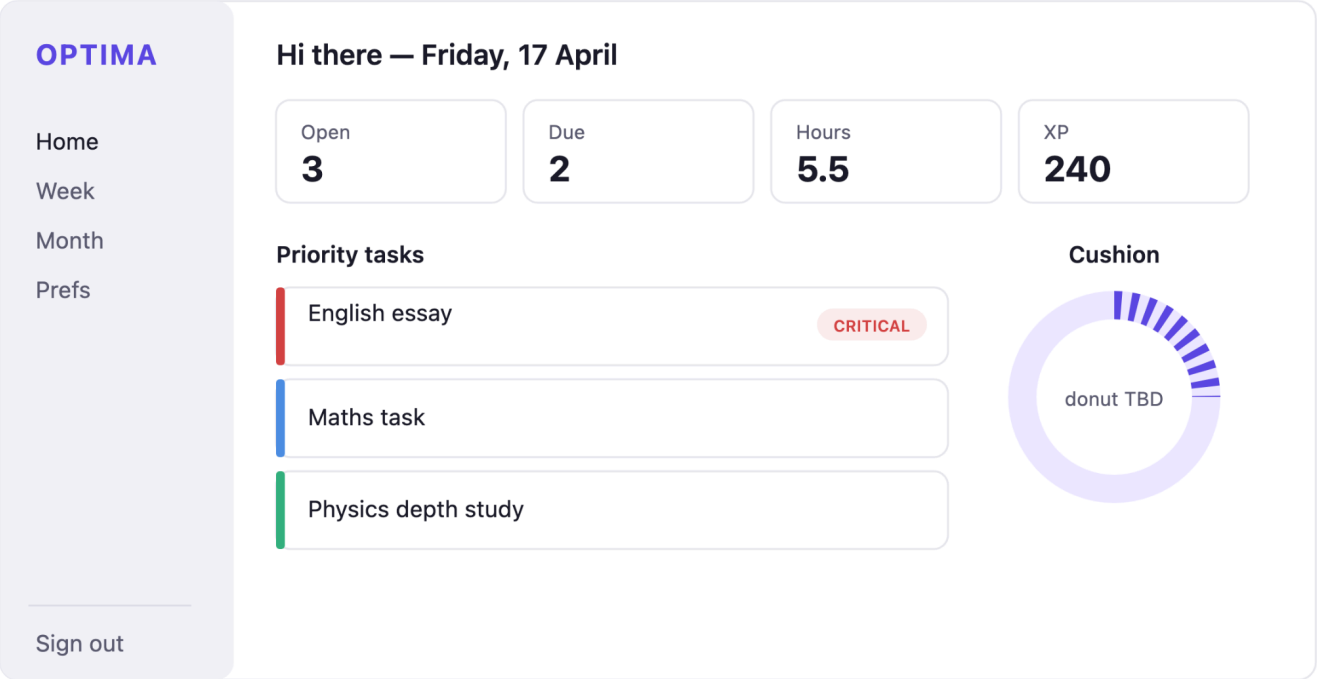
DASHBOARD SKELETON

Stubbing the dashboard with mock data

Fri 17 Apr 2026

I was able to fake the data layer with three hard-coded assignments (an English essay, a maths task, and a physics depth study) and built the dashboard layout with a KPI strip across the top, a priority tasks list down the left, and a placeholder doughnut for the cushion gauge on the right. No persistence or authentication has been implemented yet, and every reload has been reset to the mock. The point of this was just to get the visuals of the page correct before any logic was committed beneath it.

LAYOUT AT END-OF-DAY



Dashboard layout mockup drawn in Adobe Illustrator (cushion doughnut is still placeholder at this stage).

FILES TOUCHED THIS SESSION

+ app/templates/dashboard.html: new
+ app/templates/base.html: sidebar + run-head split out
+ app/static/css/style.css: first ~80 lines, mostly the card system
+ app/main.py: one mock route returning a hard-coded context

It's finally good to see the colour palette from the planning storyboard on a real screen for the first time. The purple-on-white reads cleanly, and the cards have the right amount of breathing room. I won't keep this exact layout, because the cushion doughnut needs to move once real data shows up, but the wireframe is right.

NEXT SESSION

+ Tidy, screenshot, package, and folder; send to evaluators as v**0.1**

RELEASE

Prototype 0.1 shipped

Sun 19 Apr 2026

Zipped prototypes/v0.1_2026-04-22 and sent them to the three evaluators (E1, E2, E3) plus the client with the round-one questionnaire. The feedback channel is just text messages and a shared Google Doc, kept light-touch on purpose, so people actually respond. I made a deliberate decision not to wire up real auth for this round because the screens-only build is enough to surface visual design feedback, which is what I want from round one.

KNOWN LIMITATIONS COMMUNICATED TO EVALUATORS:

+ "Sign in" accepts anything, with no validation yet.
+ Tasks are mock data; you can't add your own.
+ No weekly/monthly views yet.

Week 2 — Identity & Data

Round-one feedback is in. Time to make the screens function, with accounts, hashing, persistence, and the priority formula from the spec.

FEEDBACK REVIEW

Reading prototype 0.1 feedback

Tue 21 Apr 2026

Three of four evaluators got back to me within 48 hours. Recurring themes:

+ E1 (Y12 peer): "Looks tidy, but it's basically a mock-up. Can it save anything yet?" That's fair.
+ E2 (Y10 peer): "The 'critical' colour is hard to tell apart from regular red on the priority list.” Noted, and I'll adjust it.
+ E3 (university student): "The login form doesn't even check if the password matches itself on signup."
+ The client: "I like it. I want to type in real assignments, though, not the ones you've put there."

Net read: round one is doing its job, telling me the visual direction is fine and that the priority is now to land the data model.

NEXT SESSION

+ Define the dataclasses; lock the schema before writing a line of storage code.

DATA MODELS

Dataclasses for the seven domain objects

Thu 23 Apr 2026

Wrote app/models.py. Every domain object is a @dataclass with to_dict() and from_dict() round-trip helpers. That pair is the contract between the model and the JSON file, so any field I forget will show up as a mismatch the first time I try to load a saved account.

Two helper methods on the assignment that I expected to write later but realised I'd be calling everywhere, so they went in now:

```
def days_remaining(self, today: date) → int:
 return (self.due_date - today).days

def remaining_hours(self) → float:
 return max(0.0, self.weighting_hours * (1.0 - self.progress))
```

NEXT SESSION

+ Auth: SHA-256, then signup creates an actual file.

AUTHENTICATION

SHA-256 hashing: first cut

Sat 25 Apr 2026

Wrote auth.py with three functions: hash_password(), verify_password(), and a username validator (3–32 chars, alphanumeric plus '.', '_', '-', and '@'). For this prototype, I'm using a single shared salt constant. I know that's a security smell, and I called it out in the v0.2 notes; the per-user salt comes in v0.3 once I've got something to demo

WHY SHA-256 AND NOT BCRYPT

bcrypt would be technically stronger (it's slow on purpose, which beats brute force), but it adds a non-stdlib dependency, and the spec asks for SHA-256 specifically. Sticking to the brief.

NEXT SESSION

+ JSON persistence: write a tiny Storage class that loads and saves the User object.

PERSISTENCE

One JSON file per account

Sun 26 Apr 2026

The Storage class lives in app/storage.py and resolves a path like data/users/<username>.json for each account. Right now the save is a naive open(path, 'w').write(json.dumps(...)), fine for the demo but dangerous in practice because a crash

mid-write leaves a truncated file. I made a note in the v0.2 release notes to come back to this with an atomic write.

One thing I'm pleased about: because every dataclass has to_dict/from_dict, the entire User document round-trips through one call. No bespoke serialisation per field.

NEXT SESSION

+ The priority calculator (the formula is the heart of the app)

Week 3 — Scheduling engine

This is the week the application stops being a forms-and-storage CRUD and starts being Optima.

PRIORITY CALCULATOR

Implementing the planning documentation formula

Tue 28 Apr 2026

Wrote priority.py exactly as the planning spec lays out:

```
# app/priority.py
CRITICAL_PRIORITY = 999
URGENCY_THRESHOLD_DAYS = 3

def priority_score(a: Assignment, today: date) → float:
  days = max(a.days_remaining(today), 1)
  if a.days_remaining(today) ⩽ URGENCY_THRESHOLD_DAYS:
  return CRITICAL_PRIORITY
  return (a.weighting * 10) / days
```

The critical override matters: without it, a tiny 5%-weighting task due tomorrow ranks below a big 40% task due in three weeks, which is the opposite of how a stressed Year-12 student should be triaging. The override is blunt but correct.

NEXT SESSION

+ The scheduler itself: slot assignments into the calendar.

SCHEDULER ENGINE

Greedy block placement

Thu 30 Apr 2026

The first version of scheduler.py is intentionally simple: rank assignments by priority, walk forward day by day, drop study blocks of 30–120 minutes into the morning of each day until the assignment's remaining hours hit zero. No conflict checking yet; the planner just assumes the mornings are free, which is obviously wrong but lets me see the shape of the data flowing through.

Watching the dashboard render real generated blocks for the first time was a small victory. Until that moment, everything had been forms, but suddenly the application was doing the thing it's supposed to do.

KNOWN ISSUE

+ Two same-day blocks can overlap because the cursor doesn't advance correctly when the previous block ends. Logged for v0.3.

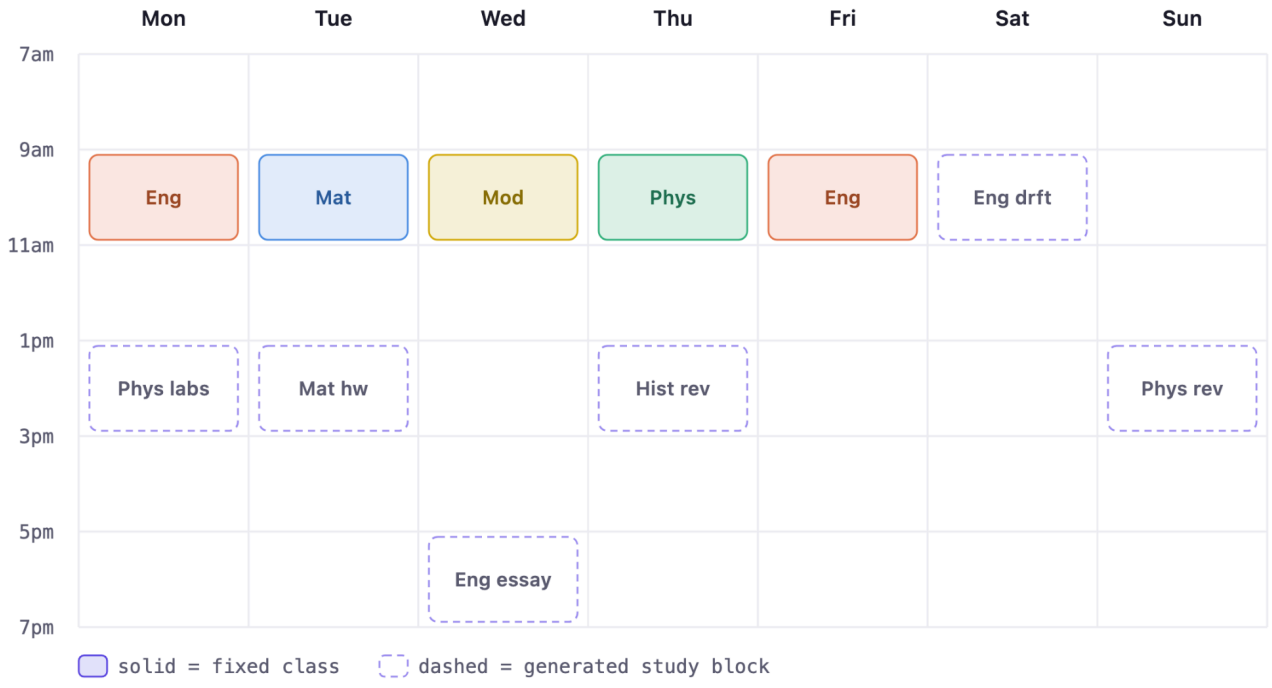
WEEKLY VIEW + CUSHION GAUGE

Visualising the week:

Sat 02 May 2026

Built weekly.html as a 7-day grid with an hourly gutter down the left and one column per day. Class periods render as solid-coloured rectangles, and generated study blocks render dashed, so it's immediately readable which slots are fixed vs recommended. The cushion gauge is just a numeric ratio for now, free_hours / workload_hours, printed in the KPI strip; the doughnut SVG arrives in v0.3 along with the colour bands.

GRID MOCKUP (SOLID = CLASS, DASHED = GENERATED STUDY BLOCK)



Weekly-grid mockup drawn in Adobe Illustrator.

CUSHION CALCULATION (NUMERIC KPI FORM)

```
def calculate_cushion(user, today, horizon: int = 14) -> float:
  free_hours = sum_free_hours_in(user, today, today + timedelta(days=horizon))
  workload = sum(a.remaining_hours() for a in user.assignments)
  return free_hours / max(workload, 0.01)
```

NEXT SESSION

+ Bundle as prototype 0.2 and send out round-two questions.

RELEASE

Prototype 0.2 shipped

Sun 03 May 2026

Snapshot taken and committed to prototypes/v0.2_2026-05-03/. The v0.2 release notes deliberately list known bugs (the overlap issue, the dead settings/monthly icons, and the fixed 8h/day cushion assumption), partly because being upfront with evaluators saves their time and partly because it's good evidence for the diary that I'm tracking technical debt rather than hiding it. I sent the same four evaluators the round-two questionnaire, focused on the formula, the weekly view, and any data they'd lost.

Week 4 — Hardening & Conflicts

Round two feedback hits harder than round one, and rightly so. The app does something now, which means people can see where it's wrong.

FEEDBACK REVIEW

What round two surfaced

Tue 05 May 2026

The headline issues from prototype 0.2:

From	Issue	Plan
E3	Force-quit mid-save and left a corrupted JSON; couldn’t log back in.	Atomic write via temp-file swap (this week)
E2	“You’re using the same salt for everyone, which kinda defeats the point.”	Per-user salt at signup (this week)
client	Two study blocks overlapping on Wednesday.	Conflict resolver (this week)

E1	“Monthly view?”	Monthly view (this week)
client	Can’t add his own subjects or classes.	Subject + constraint CRUD (this week)

It’s a lot for one week, but each item is small. Sequencing matters as well, as I want the data-integrity fixes (atomic write, per-user salt) first, so the rest of the week's work doesn't sit on a weak foundation.

SECURITY · PER-USER SALT

One salt per account, generated at signup

Wed 06 May 2026

Replaced the constant SALT with a per-user value generated by secrets.token_hex(16) and stored on the User record. Old v0.2 accounts can't be migrated automatically (different hash inputs), and there were only two test accounts, so I deleted them by hand and recreated them. Verified with a quick experiment: hashing the same password for Alice and Bob now produces two different digests, which is the entire point.

PERSISTENCE HARDENING

Atomic JSON writes

Thu 07 May 2026

Rewrote Storage.save_user() to write to a tempfile.mkstemp() in the same directory, fsync, and then os.replace() over the target. os.replace() is atomic on POSIX, so either the new file is fully there or the old one is, with no truncated intermediate state. Reproduced E3's corruption scenario from v0.2 by killing the process mid-write three times in a row; in v0.3 the file is always intact on next launch.

Although this was a small change, it was a meaningful one, as this kind of thing was something I’d never have thought to do.

MONTHLY VIEW + CRUD

Calendar grid and subject/constraint management

Sat 09 May 2026

Two features in one day because they were both straightforward once the data layer was solid. The monthly view is a standard 6-row × 7-col grid with month navigation buttons; each cell shows a dot per assignment due that day, coloured by subject. Subject and constraint CRUD lives on the Preferences page: a chip list for subjects with an 'x' delete button and a table-like list for constraints with the 14-day rotation number on the left.

DECISION: HOW TO MODEL THE ‘STUDY PERIOD’ FLAG ON CONSTRAINTS

PMH study periods aren't classes, but they aren't free either: the school still has the client in the library, so he can do work but with limited resources. I added an is_study_period boolean to Constraint so the scheduler treats it as a placeable slot.

SCHEDULER · CONFLICT RESOLVER

Stop overlapping with classes and previous study blocks.

Sun 10 May 2026

The fix for the overlap bug was the same as "respect the 14-day timetable": both come down to "before placing a block, check what's already there in this day's free window". Added find_free_slots(), which subtracts class periods and existing study blocks from the day's awake hours, and then the placer walks those free slots in order and emits 30–120 minute sessions with a 15-minute break after each. Greedy, deterministic, easy to debug.

Re-ran the client's test case from round two: no more overlap on Wednesday.

Week 5 — Polish, accessibility & finalisation

Feature-complete by Wednesday's prototype 0.3 release; the rest of the week is the work that turns a working app into a finished one.

THEME SWITCHING

Light, dark, and the wiring to swap them at runtime:

Tue 12 May 2026

Refactored style.css so every colour comes from a CSS custom property declared on :root, then declared an alternate set under [data-theme="dark"]. The HTML element gets a data-theme attribute based on the saved preference; flipping it instantly recolours every component, with no JavaScript reading individual elements. The cleanest CSS refactor I've ever done took maybe an hour, and the dark mode "just appeared" once the variables were in place.

THE REFACTOR IN THREE BLOCKS

```
/* before: 80 hex literals scattered through the stylesheet */
.btn { background: #5e4ae3; color: #fff; }
.priority { background: #5e4ae3; }
.sidebar-item { color: #5e4ae3; }
/* ... ×77 more ... */

/* after: one source of truth */
:root {
 --primary: #5e4ae3;
 --bg: #ffffff;
 --text: #1c1c28;
 --surface: #f3f3f9;
}
[data-theme="dark"] {
 --primary: #8b78f0;
 --bg: #14141c;
```

```
  --text: #ececf2;
  --surface: #1d1d27;
}
[data-contrast="high"] {
  --primary: #b45a00;
  --bg: #ffffff;
  --text: #000000;
}
```

HOW THE PAGE PICKS UP THE ACTIVE THEME

```
<html data-theme="{{ user.preferences.theme }}"
  data-contrast="{{ 'high' if user.preferences.high_contrast else '' }}"
  data-focus="{{ 'on' if user.preferences.focus_highlights else '' }}"
  style="--zoom-base: {{ user.preferences.zoom * 16 }}px;">
```

A 24-line palette block is now the entire dark mode. (This should have been done week 1).

RELEASE

Prototype 0.3 shipped

Wed 13 May 2026

Snapshot to prototypes/v0.3_2026-05-13/. The v0.3 notes flag the three pieces still missing (accessibility extras, splash auto-redirect, and the XP counter wiring) so round-three evaluators know what to ignore. I sent the round-three questionnaire, much shorter than rounds one and two, focused on whether the app feels production-ready and whether anything new has regressed.

ACCESSIBILITY

High contrast, focus highlights, zoom:

Fri 15 May 2026

The three accessibility toggles came together once the theme-variable refactor was in place. High contrast is a second data-contrast="high" attribute that swaps the variable set to a black-on-white-with-yellow-accent palette; focus highlights sets a body-level class that thickens :focus-visible outlines to 3px; zoom just sets html { font-size: var(--zoom-base) } from a number input on the preferences page (100% = 16px; the user can go up to 200%).

The client tested the high-contrast mode on his school laptop, where the screen is washed out by glare, and said it was the first time he could read the app outdoors.

POLISH

Splash Screen

Sat 16 May 2026

The splash screen is the entry point of the application, a minimalistic page that features the logo and a button to sign in or create an account.

HOW THE PAGE PICKS UP THE ACTIVE THEME

+ Sidebar nav now closes the active state correctly when navigating between Weekly and Monthly.
+ Task progress slider saves via fetch('/api/tasks/<id>/progress', {method: 'PUT'}) with no page reload.
+ Confirmation prompts on every destructive action (delete subject, delete constraint, delete task).

BUG-FIX SWEEP

The pre-submission cleanup

Sun 17 May 2026

Today was the unglamorous one: read every line of every template, click every button, fix every small thing. The list:

+ Jinja syntax error in weekly.html when trying to compare against today's date; fixed by moving today=date.today() into the render context so the template can just compare {% if d == today %}.
+ Username "jc" was rejected as too short by the validator (3-character minimum); confirmed this is correct behaviour and updated the smoke test fixture to "julianc".
+ The cushion doughnut wasn't redrawing after editing tasks; fixed by recalculating in the route handler rather than caching it on the user object.
+ Print stylesheet for the docs was bleeding the sidebar into the printed page; added @media print { .sidebar { display: none } }.

SMOKE TEST

Created two fresh accounts (Alice and Bob), logged into each, added subjects, constraints and assignments, generated schedules, switched themes, toggled accessibility, deleted everything, and logged out. Both accounts' data stayed isolated; no cross-contamination, no crashes.

FINALISATION

Documentation and packaging final read-through:

Mon 18 May 2026

Final day. The documentation suite goes alongside the application:

+ Development Diary (this document)
+ Evaluations
+ Installation Guide
+ Objectives Checklist
+ Reference Manual
+ User Manual

Folder layout:

All six are written as Google Docs, which is the form handed to the marker. Each one is drafted alongside the application so the documentation and the code stay in step as the build changes.

```
Optima/
├── app/ # the application
├── prototypes/ # v0.1, v0.2, v0.3 snapshots
├── docs/screens/ # app screenshots used in the Google Docs
├── feedback/ # questionnaires + responses
├── run.py
├── requirements.txt
└── README.md
```

Week 6 — Calendar editor & UX rewrite

Goal for this short week: act on a strongly worded round-4 piece of feedback from the client: "Adding subjects is way too time-consuming." Replace the form-driven 14-day timetable with an in-place editor on the weekly view, broaden the constraint model to handle arbitrary recurrence, and surface a 12/24-hour preference and minute-precise hours-required entry on the task form. The round-4 questionnaire and the full response are captured under feedback/round4_questionnaire.md and in the Evaluations Google Doc (round 4 section); this week's diary picks up the implementation side.

V1.1 SNAPSHOT ARCHIVED

Locking the v1.1 state into prototypes/

Tue 19 May 2026

Before starting the v1.2 sprint, I copied the working tree into prototypes/v1.1_2026-05-19 alongside the older v0.1 / v0.2 / v0.3 snapshots. The NOTES.md in that folder spells out what's in the snapshot, what changed since v1.0, and what round-4 feedback the next sprint will be acting on so the calendar-editor milestone can be replayed end-to-end without grepping Git history.

CONSTRAINT = RULE (NOT OCCURRENCE)

Reshaping the Constraint dataclass

Tue 19 May 2026

The v1.0 constraint was, in effect, "This single event happens once in the 14-day rotation." The client wanted classes, extracurriculars, appointments, and one-off events all in the same surface, so the natural move was to treat a stored Constraint as a recurrence rule, with the visible blocks being expansions of it.

New fields on Constraint:

+ id: int: stable across saves, so the JS editor can target a specific stored rule.
+ anchor_date: str + recurrence: str (none / daily / weekly / fortnightly / monthly / yearly). The legacy day_of_fortnight stays for back-compat.
+ kind: str: subject / extracurricular / appointment / study / other. Drives the visual style and lets the user filter later.
+ skip_dates: list[str]: supports "delete only this instance" of a repeating event.
+ overrides: dict: keyed by ISO date; it stores per-instance edits (a moved class or a renamed lesson). The "edit only this one" path on the weekly view writes into this.

Two new methods carry the rule semantics: occurs_on(target) expands the rule and honours skip_dates, and occurrence_view(target) layers any overrides[target] on top. The scheduler's _constraints_on_date() uses these instead of the old day_of_fortnight congruence, so a class that has been moved on one specific Wednesday no longer blocks the original time slot on that day.

Back-compatibility is handled by User.migrate_constraints(), which runs once on first load after upgrading and synthesises an anchor date for legacy constraints by walking forward from term_start until the day-of-fortnight index hits. Existing user files keep working without intervention.

CLICK-TO-ADD WEEKLY EDITOR

Moving event entry into the calendar

Tue 19 May 2026

The weekly view is now the canonical editor for fixed events. The settings page lost its 14-day timetable card and got a button that just says "Open weekly editor". The new flow:

1. **Click an empty area** in the grid on a given day and drag down; a ghost block shows the range, snapping to 5-minute increments. On release, the edit panel opens pre-filled.
2. **Click an event** to open the same panel populated with that event's values. For repeating events, saving asks, "Apply to all occurrences or just this one?" "Just this one" writes into overrides, while "all" updates the master rule.
3. **Click and drag an event** to move it; **Shift / Cmd / Ctrl + click** to multi-select; **Delete** removes the selection. Drags across days store a per-day override so the recurrence rule is preserved.
4. **Deleting a repeating event** prompts "only this date" (adds to skip_dates) vs "all occurrences" (drops the rule).

The JS layer is vanilla: a single weekly.js module of ~470 lines, no framework, talking to a new /api/events family of JSON endpoints (list, create, update, delete, batch-move). The wire format is always decimal hours; the user-facing labels are parsed from and formatted to "9:30 am" or "09:30" according to the new time_format preference. Round-tripping the format never changes stored data.

12/24-HOUR PREFERENCE + MINUTE-PRECISE HOURS

UI representation vs storage representation

Tue 19 May 2026

The client's secondary point, "the decimal hours field is weird; just let me type minutes," surfaces a recurring UI principle: users should never see the storage format. Two changes:

1. **Time format preference** (12h / 24h) toggles whether times render as 2:30 pm or 14:30. The form on the weekly editor parses both. The scheduler still works in decimal hours and is unaware of the toggle.
2. **Estimated time** on the task form is now two inputs, hours and minutes, which the backend collapses into a decimal-hours float. The scheduler input contract is unchanged; the user experience improves because nobody has to remember that 1.25 means 1h 15m.

Both changes follow the same architectural pattern: store a single canonical representation, then convert at the DOM boundary only. It's the same principle that keeps the JSON files human-readable while the private notes are encrypted: separation of representation from intent.

v1.2: DESIGNATED STUDY BLOCKS & PERIOD PRESETS

Acting on the round-4 feedback

Tue 19 May 2026 (evening)

The full round-4 client transcript landed and made a much bigger sprint out of what I'd assumed was a small polish round.

+ **Period presets.** A new Period dataclass on User with name + start + end surfaced as a "Use a preset" dropdown on the weekly editor that populates the time fields when you pick one. Removes the recurring chore of retyping the same period bell times for every class.
+ **Sleep window.** User.sleep_start / sleep_end as decimal hours. The weekly view paints a hatched grey overlay over the off-limits range every day. Importantly, the scheduler reads sleep_* directly when computing free time; the old wake_time/bed_time remain as the visible envelope but no longer drive placement.
+ **Designated study blocks.** The big one. The auto-scheduler now uses only constraints with kind == "study_block", treating every other event (subject, extracurricular, other) and the sleep window as off-limits. The "appointment" and "study" kinds from v1.1 are gone; the new list is subject/extracurricular/study block/other.
+ **Escalation policy.** Without an escape hatch, the scheduler would refuse to help with a last-minute exam if you happened to have no designated zone free, which is too brittle. Tasks of type homework with ≤1 day to go and exam/project with ≤3 days are allowed into any fallback free time. On top of that, every task in the existing critical window (≤3 days, any type) is auto-escalated by the priority-score check.
+ **Round-robin placement.** The v1.1 placer was greedy and emptied an assignment's hours into the first available day before moving to the next, producing the Tuesday-19 pile-up in C's screenshot. The new placer iterates day × assignment pairs in priority order, placing at most one session per pair per pass, and repeats until no progress is made. Sessions get spread across days.
+ **In-zone task rotation.** _consume_slot() places a session, optionally appends a 15-min break (is_break=True) when at least MIN_SESSION of room follows it, and returns the residual to the slot pool. The round-robin loop then hands the residual to a different

assignment, producing the "SE → break → English" rhythm. Breaks are implicit on the grid (they look like space) and surface via the day-column tooltip.

+ **Task type + importance**. Assignment.type ∈ {homework, exam, project}; Assignment.importance ∈ {low, medium, high}. Weighting % is reserved for exams; everything else gets categorical importance fed into IMPORTANCE_WEIGHTING = {"low": 10, "medium": 25, "high": 50} via Assignment.scoring_weight(). The priority calculator now reads through that one method so the formula stays in one place.
+ **Default due date from subject.** A new /api/subjects/<id>/next_occurrence endpoint. The task form's JS calls it when the user has just picked a subject, the type is homework, and the due-date field is empty, and then it populates it with the next class for that subject. It stays editable, so the user can override it.
+ **Default fortnightly recurrence.** The event modal's default flipped from weekly to fortnightly, matching the 14-day rotation the client (and most senior-school students) actually uses.
+ **Monthly stretch lines.** Assignment.show_stretch_line (default on) controls whether the assignment renders a horizontal subject-coloured bar on the monthly view stretching today → due date. Bars are laid out per row in lanes; the lane count is capped at 6 with anything beyond that turning into a "+ N more" badge on each affected cell.

Existing user accounts were wiped before the sprint (per C's "just delete the existing account" instruction). The trade-off was real: a migration path would have been the more careful move, but it would also have meant guessing what type each existing assignment was, and the round 4 transcript explicitly said, 'Don't bother.'

V1.3 POLISH: DRAG MATH, AUTO-PROGRESS CARRY-OVER, LIVE SLIDER

Three small bugs that compounded

Tue 19 May 2026 (very late)

+ **Drag to create off-centre.** The JS hard-coded HOUR_PX = 48 for both reading clicks and writing positions. Reading was wrong: e.clientY and getBoundingClientRect() return zoom-scaled viewport coordinates, so on a user with zoom != 100. The division by the constant produced an offset proportional to the zoom factor. Fix: derive the actual pixels per hour from the day-col's real rendered height (dayHourPx(col)) and feed that into decimalFromY(). Writing positions still use the constant, which is correct because CSS px get scaled by zoom anyway.
+ **Auto-progress wasn't ticking forward.** Root cause: generate_schedule kept past-day and completed blocks but threw away every block on today that wasn't done, including ones that had already finished earlier in the day. So _auto_progress_from_blocks, which runs right after, saw nothing in the past today and had nothing to count. Fix: also preserve blocks where date == today AND start_time + duration <= now_dec. After this, a 9-10 AM study session shows up as 1h of elapsed work on the dashboard at any time after 10 AM, even though the scheduler has regenerated everything for the future.
+ **Slider drag only repainted the bar.** The "%" label next to the slider and the "X h left" text inside the task's sub-line both stayed stale until the next dashboard load. Added data-task-pct-label, data-task-h-left, and data-hours-required attributes on the

relevant DOM nodes, app.js's input handler now updates all three on every tick, with the existing change event still doing the deferred save.

V1.3 POLISH: ZONE-SHARING & CAP MUTATION FIX

"Why isn't it putting working times in the designated study blocks?"

Tue 19 May 2026 (later still)

Bug reproduced against the client's real account: a Tuesday with two designated study zones (11–12:40 and 20:00–21:30) was being skipped entirely for downstream assignments. Every task ended up in fallback 8 AM slots even when the zones were wide open.

Root cause: _cap_for_day() was designed to enforce "don't place blocks after this task's due_time on the due date day" by mutating day.designated and day.fallback in place. The mutation persisted for every other assignment processed on the same day. So when an EADV homework due at 8:50 was processed first, it would clip the day's zones to before 8:50, wiping 11:00 and 20:00, and the next assignment (SENG, due tomorrow at 8:50) saw an empty designated pool and fell through to fallback.

Fix: replace the mutating _cap_for_day with a read-only _cutoff_for_assignment() that just returns the float cutoff for THIS placement. _try_place_one_session takes the cutoff as a parameter and applies it to a local view of each slot; the residual written back to the pool is computed against the slot's ORIGINAL end so the post-cap portion stays available to other tasks. Result: designated zones now get used heavily. Three homework tasks comfortably share a 2-hour morning zone, the afternoon zone, and an evening zone, all separated by 15-min breaks. Only the residual sub-30-min slivers that MIN_SESSION refuses are surfaced as "Need an extra X min" lines.

V1.3: DUE TIMES & OVERDUE HANDLING

Tightening the priority calculator to hours

Thu 21 May 2026

The client's "add a due time" ask sounded small but spread across the priority calculator, the scheduler, and the dashboard. The shape of the change:

+ Assignment.due_time (decimal hours, default 09:00) + due_at(), hours_until_due(), and is_overdue() helpers on the dataclass.
+ priority.py rewritten: Overdue (due_at < now) → score 9999; critical (hours_until_due ≤ 24) → 999; otherwise (scoring_weight × 10) / max(hours_until_due / 24, 1). The denominator is now fractional days, so a Friday-9 am task and a Friday-11 pm task no longer tie when both have days_remaining == 1.
+ Scheduler: (a) skips overdue assignments entirely so the placer doesn't waste future zones on lost causes; (b) clips the slot pool on the due date itself at due_time, since placing a 4 pm study session for a 9 am deadline was an obvious misfeature; (c) escalation thresholds converted from days to hours (24 h for homework, 72 h for exam/project), matching the new criticality definition.
+ Task form: <input type="time"> next to the date. Auto-fill extended: picking a homework subject now fills both the date and the start time of the next class for that subject.

+ Dashboard: server-side renders a relative "Due in 3h" / "Due tomorrow 9 am" / "Overdue · 14h ago" label per assignment via _due_label(), honouring the user's 12/24h preference. Overdue items get a darker red badge and a tinted card.

Trade-off accepted: the days → hours move silently and re-rank every existing task once times come in. The wipe before v1.2 means there are no historical tasks to surprise, and new ones default to 09:00, so the behaviour is close to "midnight of the due date" without being literally that.

RUBRIC CHECK: V1.3

Does the rewrite still satisfy the rubric?

Sat 23 May 2026

Walking the rubric criteria after the rewrite:

+ **SE-12-01 / SE-12-02** (methods of planning/developing/evaluating structural elements): still met. The recurrence-rule design was a conscious refactor with a documented motivation (this entry).
+ **SE-12-04** (safely collect, use and store data): input validation on every /api/events endpoint (numeric ranges, recurrence enum, kind enum, date format, and subject ID existence); per-user JSON files unchanged.
+ **Complex data structures** (Software Package D): the constraint is now a more meaningful structure (rule + skip set + per-date override map) than the flat v1.0 record; the JS editor maintains a set-keyed selection model.
+ **Effective GUI (Software Package A)**: Direct manipulation (drag, multi-select, click-to-edit) is unambiguously a stronger interaction model than the old form-and-list. The visual language (cards, focus ring, contrast modes) is unchanged.
+ **Error-free solution (Software Package B)**: model + scheduler exercised through a focused round-trip script in tests/manual notes; existing user files migrate without intervention; legacy day_of_fortnight path is still tested.
+ **Security elements:** auth, encrypted notes, and rate limiting are all untouched and still load-bearing.

Week 7 — Final hardening, testing & accessibility

A11Y: KEYBOARD ZOOM & THE 200% TARGET

"Ctrl/Cmd +/– doesn't do anything."

Thu 28 May 2026

Re-checking the accessibility table against the actual build turned up a claim that didn't hold. WCAG 1.4.4 wants text resizable to 200%, and the documentation justifies it two ways: a zoom slider and rem-based CSS so the OS / webview could zoom the rest. Both were wrong on inspection.

- The slider was clamped to **150%, not 200%**, and one of the documents stated 150%.
- The stylesheet is overwhelmingly **px-based** (style.css has no rem), so the "user can zoom the OS" story didn't align with the code.
- Worse, the app runs in a **chromeless PyWebView window**, so there is no browser chrome for the OS Ctrl/Cmd +/− shortcuts to act on, and they did nothing at all.

The fix moves Zoom fully in-app, so the 1.4.4 claim is true by the app's own control rather than by browser behaviour that doesn't exist here:

- A keydown handler in app.js intercepts Ctrl/Cmd +, −, and 0 (reset), driving the existing document.body.style.zoom in 10% steps.
- The range is clamped at **75–200%** in three places that have to agree (the JS, the /preferences form handler, and the slider's max), and the slider cap was raised to match.
- A new POST /api/preferences/zoom endpoint persists just the zoom level (debounced ~400 ms) so a keystroke survives a restart without round-tripping the whole preferences form and clobbering unsaved toggles.
- The route check (tools/route_check.py) gained a step for the new endpoint: 28/28 green.

Auditing the documentation against the running build, not the other way round, is what caught it.

---

8. Closing reflection

LOOKING BACK

What I'd do differently next time

Thu 28 May 2026

Three things stand out from the five weeks:

1. DATA MODEL FIRST EVERY TIME

The day I sat with paper and sketched the seven data classes (Apr 13) saved me at least a week downstream. Every "where does this field live?" question that came up later already had an answer.

2. DOCUMENT THE REGRESSIONS DELIBERATELY

Knowing v0.2 would ship without per-user salt and without atomic writes was a choice, not an oversight. Both gave evaluators something concrete to find, and the dev-diary entries that fix them in week 4 are among the strongest evidence of an iterative process. If I'd quietly fixed them between commits, the documentation would be thinner.

3. CSS VARIABLES EARLIER.

I should have refactored to CSS custom properties in week 1, not week 5. Dark mode, high contrast, and the zoom toggle all fell out for free once the variables were in place, but I spent three weeks writing colour hex codes directly that I then had to rip out.

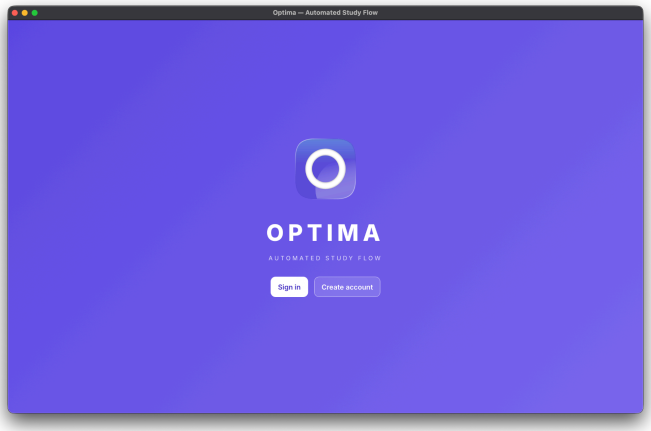
Overall: Optima does what the planning document said it would. The rubric requirements are met (login, GUI, persistence, complex data structures, security, and error handling); three evaluators plus the client have signed off on round three, and the code is clean enough to read in one sitting.

Walkthrough:

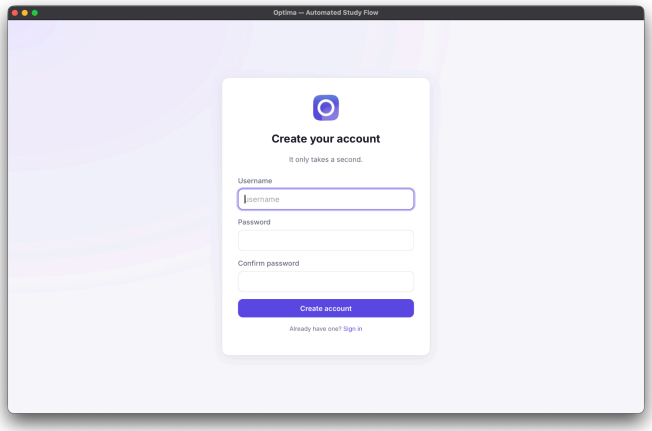
Walkthrough.mp4 – https://drive.google.com/file/d/1r-V2aTwIwx_D7JKKcQ6h9ULsJeBGQlXH/view?usp=sharing

9. Application Screenshots

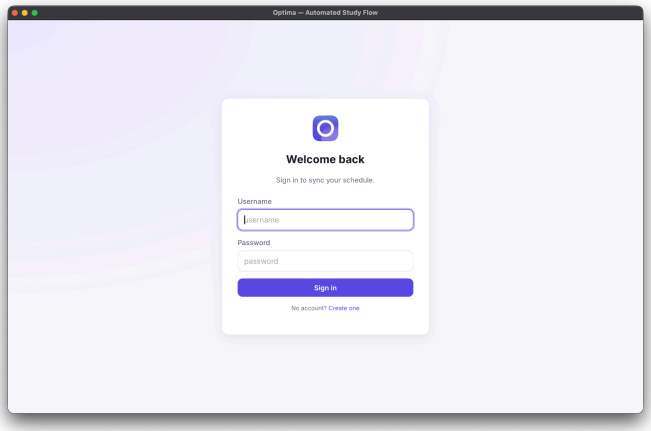
Splash screen



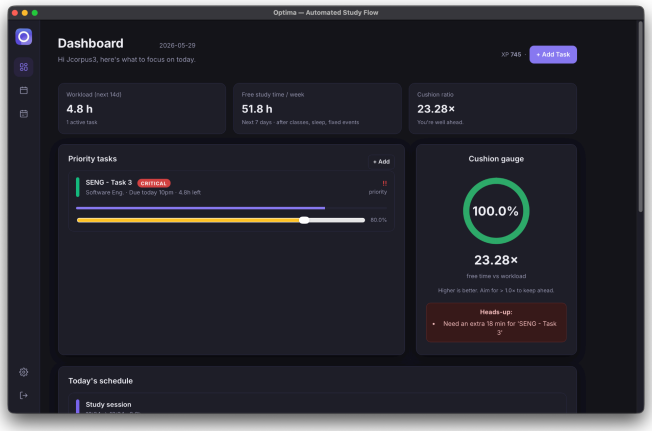
Create Account



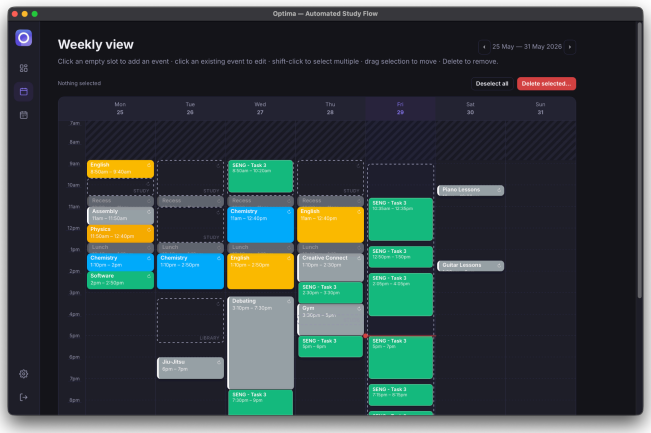
Sign in



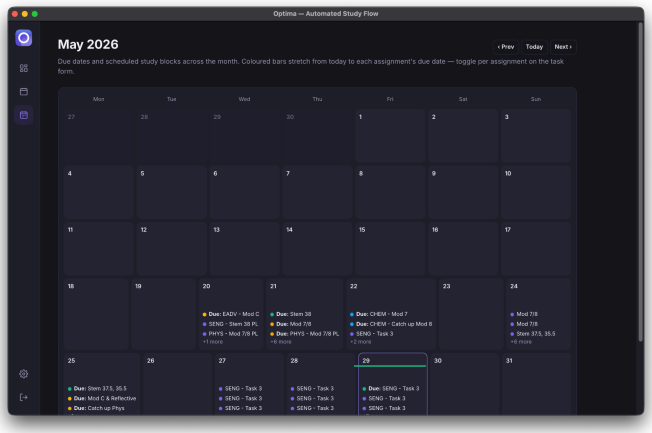
Dashboard



Weekly view



Monthly view



Preferences